VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Personal Expense Tracker

Course End Project

Submitted by

Name of the Student:	Akhil
Roll Number:	25881A05CC
Class/Section:	CSE-G

Course Facilitator

Name: _____

Designation: _____

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

Abstract ii
1 Introduction 1
2 Problem Statement 1
3 Objectives 1
4 Literature Review 1
5 System Requirements 2
5.1 Hardware Requirements 2
5.2 Software Requirements 2
6 Methodology and System Design 2
6.1 System Architecture 2
6.2 Database Design 3
6.3 Main Modules 3
6.4 Workflow 3
7 Implementation 4
7.1 Technology Stack 4
7.2 Input Validation and Data Handling 4
7.3 How to Run the Application 4
8 Testing and Discussion 4
9 Mapping of Program Outcomes and Sustainable Development Goals 5
9.1 Program Outcomes (POs) Mapping 5
9.2 Sustainable Development Goals (SDGs) Mapping 6
10 Conclusion and Future Enhancements 6
References 6

Abstract

Managing day-to-day spending can be difficult when purchases are recorded informally or not recorded at all. The Personal Expense Tracker is a lightweight web application designed to help an individual record, review and organize expenses from one place. The application is implemented using Python and Flask for application routing, SQLite for persistent local storage, and HTML and CSS for the user interface. A user can add an expense with a title, amount, category, date and optional note; view overall and current-day spending; review totals by category; search and filter transactions; delete an entry; and export records as a CSV file for spreadsheet analysis. The database is created automatically when the application starts, reducing setup effort. The application is designed for local use and demonstrates basic web development, input validation, database operations and data presentation. It can be extended with authentication, budgets, monthly analytics and cloud storage. The current version is a student project and is not intended to provide multi-user security or financial advice.

Keywords: Personal Expense Tracker, Flask, Python, SQLite, expense management, CSV export, web application.

1 Introduction

Personal budgeting starts with understanding where money is spent. When expenses are tracked in notebooks or scattered notes, it can be difficult to search previous entries, compare categories or calculate totals. A digital expense tracker can make these tasks more consistent by keeping records in a structured database and providing quick summaries.

The Personal Expense Tracker is a browser-based application that runs locally on a computer. It provides a simple interface for entering expenses and reviewing saved transactions. The project combines a Python backend, a relational database and a responsive web interface. It is intended as an educational project demonstrating how a small full-stack application can solve a practical everyday problem.

2 Problem Statement

Users may forget small purchases or find it inconvenient to calculate their spending manually. Paper records and unstructured notes do not easily support filtering, category-wise totals or export. The problem addressed by this project is to provide a simple, low-cost tool for recording personal expenses and retrieving useful summaries without requiring a paid service.

3 Objectives

- To develop a simple and responsive web application for personal expense recording.
- To store expense details persistently using an SQLite database.
- To calculate total spending, today's spending and the number of transactions.
- To group recorded expenses by category and display their relative totals.
- To support searching and filtering by text, category and date range.
- To allow users to delete records and export transactions to CSV format.
- To apply basic validation and parameterized database queries.

4 Literature Review

Common approaches to expense tracking include paper ledgers, spreadsheet templates, mobile finance applications and web-based budgeting tools. Paper records are easy to start but require manual calculations. Spreadsheets support formulas and charts but may require users to maintain their own structure. Full-featured finance applications may provide synchronization and bank integration, but can introduce account setup, privacy and service-dependency concerns.

This project focuses on a smaller local-first implementation. Flask provides URL routing and request handling with a compact Python application structure, while SQLite stores data in a local relational database without requiring a separate database server. CSV export enables further analysis in spreadsheet software. Compared with a full commercial budgeting platform, this version deliberately limits its scope and does not connect to bank accounts.

5 System Requirements

5.1 Hardware Requirements

- A desktop computer or laptop capable of running Python.
- At least 2 GB RAM recommended for normal development use.
- Keyboard, pointing device and a display.

5.2 Software Requirements

- Operating system: Windows, Linux or macOS.
- Python 3.10 or newer.
- Flask web framework.
- SQLite database engine (included with standard Python builds).
- A modern web browser and a code editor or terminal.

6 Methodology and System Design

6.1 System Architecture

The application follows a simple three-part architecture. The presentation layer is an HTML page styled with CSS. The application layer is a Flask server that validates input, processes requests and prepares summaries. The data layer is SQLite, which stores each expense as a row. The browser sends form submissions to Flask, Flask performs a database operation and then redirects the browser to the updated page.

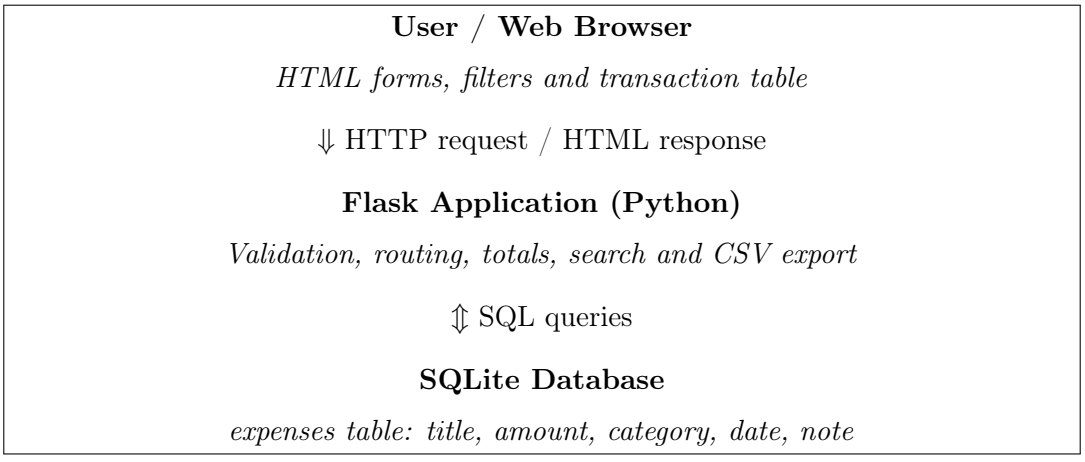


Figure 1: High-level architecture of the Personal Expense Tracker.

6.2 Database Design

The database contains an `expenses` table. The `id` field is an auto-incrementing primary key. The `title`, `amount`, `category` and `expense_date` fields store required information, while `note` is optional. The amount must be positive. The table is created automatically at application startup if it does not already exist.

Table 1: Expense table fields

Field	**Type**	**Purpose**
id	INTEGER	Primary key; uniquely identifies each record.
title	TEXT	Short description of the expense.
amount	REAL	Expense amount; must be greater than zero.
category	TEXT	Spending category selected by the user.
expense_date	TEXT	Date of the expense in ISO format (YYYY-MM-DD).
note	TEXT	Optional additional description.

6.3 Main Modules

1. **Expense entry:** accepts title, amount, category, date and optional note.
2. **Dashboard:** calculates total spending, today's spending and transaction count.
3. **Category summary:** groups records by category and displays totals with proportional bars.
4. **Search and filtering:** filters by title or note, category and date range.
5. **Delete operation:** removes a selected record after a browser confirmation.
6. **CSV export:** creates a downloadable CSV file containing the saved records.

6.4 Workflow

1. The user opens the local application in a browser.
2. The user enters expense details and submits the form.
3. The Flask route validates required fields and checks that the amount is positive.
4. A parameterized SQL statement stores the record in SQLite.
5. The page reloads and displays refreshed totals, category summaries and transactions.
6. The user may search, filter, delete a record or export the records to CSV.

7 Implementation

7.1 Technology Stack

- **Python:** backend language and data processing.
- **Flask:** handles routes, form submissions, templates and feedback messages.
- **SQLite:** local persistence for expense records.
- **HTML:** page structure, input forms and transaction table.
- **CSS:** responsive layout, cards, category bars and table styling.
- **CSV module:** generates a downloadable spreadsheet-compatible file.

7.2 Input Validation and Data Handling

The application checks that the title and date are supplied, the amount can be converted to a number and the amount is greater than zero. The selected category must be one of the supported categories. Database commands use placeholders for user-supplied values, which reduces SQL injection risk. The CSV export uses Python's CSV writer so commas and quoted text in notes are handled correctly.

7.3 How to Run the Application

1. Install Python 3.10 or newer.
2. Open a terminal in the project directory.
3. Install dependencies using `pip install -r requirements.txt`.
4. Run `python app.py`.
5. Open http://127.0.0.1:5000 in a browser.

The SQLite database file is generated locally. The database is excluded from version control so private transaction records are not unintentionally uploaded to a public repository.

8 Testing and Discussion

The following test cases define the expected behavior and should be executed on the final machine before submission. Results should be updated if actual testing produces a different outcome.

Table 2: Functional test cases

No.	Test	Expected result	Status
1	Add a valid expense	Record is saved and appears in the transaction list.	To be verified

No.	Test	Expected result	Status
2	Submit an empty title	Validation message is displayed; no record is inserted.	To be verified
3	Enter zero or negative amount	Entry is rejected with a validation message.	To be verified
4	Filter by category	Only matching category records are displayed.	To be verified
5	Search by title or note	Matching records are displayed.	To be verified
6	Delete a record	Selected record is removed after confirmation.	To be verified
7	Export CSV	Browser downloads a CSV with column headings and records.	To be verified
8	Restart the application	Previously saved records remain in SQLite.	To be verified

The application is intended to make common expense-recording tasks available from one page. Its local database avoids the need to configure an external database server. However, the current implementation does not provide user login, encryption at rest, cloud synchronization, edit-existing-record functionality or automatic bank imports. Since data is local, users should back up the database file if they need to preserve records; the file should not be committed to a public repository.

9 Mapping of Program Outcomes and Sustainable Development Goals

9.1 Program Outcomes (POs) Mapping

Table 3: Program outcome mapping

PO	Outcome area	Project relevance
PO1	Engineering knowledge	Applies programming, database and web-development concepts.
PO2	Problem analysis	Identifies difficulties in manual expense tracking and retrieval.
PO3	Design/development of solutions	Designs a working application with entry, summary and export modules.
PO5	Modern tool usage	Uses Python, Flask, SQLite, HTML and CSS.

PO	Outcome area	Project relevance
PO8	Ethics	Highlights responsible handling of personal financial records and repository privacy.

9.2 Sustainable Development Goals (SDGs) Mapping

Table 4: SDG mapping

SDG	Goal	Project relevance
SDG 4	Quality Education	Supports practical learning in programming and data management.
SDG 8	Decent Work and Economic Growth	Encourages basic financial awareness and organized expense records.
SDG 12	Responsible Consumption and Production	Helps users review spending patterns and make more informed consumption choices.

10 Conclusion and Future Enhancements

The Personal Expense Tracker presents a simple local web application for recording and reviewing individual expenses. It combines a Flask backend, SQLite persistence and a responsive HTML/CSS interface. Its planned core functions include adding records, viewing spending summaries, filtering transactions, deleting entries and exporting data to CSV. The modular structure makes the application suitable for further development and academic demonstration.

Future enhancements may include user authentication, editing saved expenses, monthly charts, budget limits and alerts, recurring transactions, automated test coverage, secure backups and cloud database support. Before using the application with sensitive real-world financial information or deploying it publicly, authentication and additional security controls should be implemented.

References

Bibliography

[1] Pallets Projects, "Flask Documentation." Available: https://flask.palletsprojects.com/. Accessed October 2026.

[2] Python Software Foundation, "Python 3 Documentation." Available: https://docs.python.org/3/. Accessed October 2026.

[3] SQLite Consortium, "SQLite Documentation." Available: https://www.sqlite.org/docs.html. Accessed October 2026.

[4] MDN Web Docs, "HTML: HyperText Markup Language." Available: https://developer.mozilla.org/en-US/docs/Web/HTML. Accessed October 2026.

[5] GitHub Docs, "About repositories." Available: https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories. Accessed October 2026.